

# DAY 14: PANDAS DATA CLEANING & DATA MANIPULATION

Complete Study Resource • 60-Day Data Science Learning Journey • Muhammad Ahmad

**Prerequisite:** Day 13 — Pandas Fundamentals

**Goal:** Learn how to take messy real-world data and turn it into clean, validated data ready for analysis and Machine Learning.

## 1. What Is Data Cleaning?

Data cleaning means finding and fixing problems in a dataset. Common problems include missing values, duplicate records, inconsistent capitalization, extra spaces, incorrect data types, invalid numbers, and inconsistent category names.

| Name  | Marks | CGPA | Department |
|-------|-------|------|------------|
| Ali   | 85    | 3.2  | CS         |
| Usman | NaN   | 3.0  | cs         |
| Hamza | 91    | NaN  | SE         |
| Ali   | 85    | 3.2  | CS         |
| Ahmad | 78    | 3.1  | CS         |

This dataset contains missing values, a duplicate record, and inconsistent department formatting.

## 2. Why Data Cleaning Matters

Data Science depends on data quality. A useful rule is: Garbage in → Garbage out. Cleaning should happen before serious analysis, visualization, or Machine Learning.

Raw Data → Inspect → Clean → Transform → Analyze → Visualize → Machine Learning

## 3. Import Pandas and Create a Messy Dataset

```
import pandas as pd
import numpy as np

data = {
    "Name": [" Ali ", "Usman", "Hamza", "Ali", "Ahmad"],
    "Marks": [85, np.nan, 91, 85, 78],
    "CGPA": [3.2, 3.0, np.nan, 3.2, 3.1],
    "Department": ["CS", "cs", "SE", "CS", "CS"]
}

df = pd.DataFrame(data)
print(df)
```

**Line by line:** pandas is imported as pd. NumPy is imported as np. np.nan represents a missing numerical value. pd.DataFrame(data) converts the dictionary into a DataFrame.

## 4. Inspecting a Dataset

```
print(df.head())
print(df.tail())
print(df.shape)
print(df.columns)
print(df.dtypes)
df.info()
```

head() shows the first rows. tail() shows the last rows. shape returns rows and columns. columns lists column names. dtypes shows data types. info() summarizes structure and non-null values.

## 5. Missing Values

```
print(df.isnull())
print(df.isnull().sum())
print(df.notnull())
print(df[df["Marks"].notnull()])
```

isnull() returns True for missing values. isnull().sum() counts missing values by column. notnull() does the opposite and identifies available values.

## 6. fillna(): Filling Missing Values

```
# Fixed value
df["Marks"] = df["Marks"].fillna(80)

# Mean
mean_marks = df["Marks"].mean()
df["Marks"] = df["Marks"].fillna(mean_marks)

# Median
median_marks = df["Marks"].median()
df["Marks"] = df["Marks"].fillna(median_marks)

# Mode
df["Department"] = df["Department"].fillna(
    df["Department"].mode()[0]
)

# Fixed category
df["Department"] = df["Department"].fillna("Unknown")
```

Mean is the average. Median is the middle value after sorting and is often less affected by outliers. Mode is the most frequent value and is commonly useful for categorical data.

## 7. dropna(): Removing Missing Rows

```
clean_df = df.dropna()

# Only require Marks to be present
clean_df = df.dropna(subset=["Marks"])
```

dropna() removes rows containing missing values. subset lets you specify which column or columns should be checked. Do not automatically delete every missing row because useful information may be lost.

## 8. Duplicate Rows

```
print(df.duplicated())
print(df.duplicated().sum())

df = df.drop_duplicates()
```

duplicated() marks duplicate rows with True. sum() counts them. drop\_duplicates() removes duplicate rows. Verify that repeated rows are actually errors before deleting them.

## 9. Fixing Column Names

```
df = df.rename(columns={
    "Student Name": "student_name",
    "student marks": "marks",
    "CGPA %": "cgpa",
    "Department Name": "department"
})

df.columns = df.columns.str.strip().str.lower()
```

rename() changes selected column names. The str.strip().str.lower() pattern removes surrounding spaces and converts column names to lowercase.

## 10. Cleaning Text

```
df["Name"] = df["Name"].str.strip()
df["Department"] = df["Department"].str.upper()
df["Name"] = df["Name"].str.title()

# Chained version
df["Name"] = df["Name"].str.strip().str.title()
df["Department"] = df["Department"].str.strip().str.upper()
```

strip() removes extra spaces. upper() converts to uppercase. lower() converts to lowercase. title() converts words to title case. Chaining lets you apply multiple cleaning operations.

## 11. Replacing Inconsistent Values

```
df["Department"] = df["Department"].replace({
    "Computer Science": "CS",
    "computer science": "CS",
    "C.S.": "CS"
})
```

replace() is useful when different labels represent the same category.

## 12. Data Types and Conversion

```
print(df.dtypes)

# astype
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
df["Marks"] = df["Marks"].round().astype(int)

# to_numeric
data = {"marks": ["85", "90", "invalid", "78"]}
df2 = pd.DataFrame(data)

df2["marks"] = pd.to_numeric(
    df2["marks"],
    errors="coerce"
)

print(df2)
```

astype() converts a column to a specified type. pd.to\_numeric() converts values to numeric form. With errors='coerce', values that cannot be converted become NaN. This is very useful for dirty real-world numeric data.

## 13. Detecting Invalid Values

```
# Marks should be between 0 and 100
invalid_marks = df[
    (df["Marks"] < 0) |
    (df["Marks"] > 100)
]
print(invalid_marks)

# CGPA should be between 0 and 4
invalid_cgpa = df[
    (df["CGPA"] < 0) |
    (df["CGPA"] > 4)
]
print(invalid_cgpa)
```

The | operator means OR. Parentheses are important around each Pandas condition.

## 14. Filtering and Sorting

```
high_performers = df[df["Marks"] >= 80]

cs_students = df[
    df["Department"] == "CS"
]

result = df[
    (df["Department"] == "CS") &
    (df["Marks"] >= 80)
]

df = df.sort_values(
    by="Marks",
    ascending=False
)
```

Use & for AND and | for OR in Pandas. Use parentheses around each condition. sort\_values() sorts rows by a selected column.

## 15. Creating New Columns

```
import numpy as np

df["status"] = np.where(
    df["Marks"] >= 50,
```

```

        "Pass",
        "Fail"
    )

    df["cgpa_percentage"] = df["CGPA"] / 4 * 100

```

`np.where(condition, true_value, false_value)` creates conditional values. The second expression creates a percentage from CGPA.

## 16. Complete Student Data Cleaning Project

```

import pandas as pd
import numpy as np

data = {
    "Name": [" Ali ", "Usman", "HAMZA", "Ali", " Ahmad "],
    "Marks": [85, "90", "invalid", 85, 78],
    "CGPA": [3.2, 3.0, None, 3.2, 3.1],
    "Department": ["CS", "cs", "SE", "CS", " cs "]
}

df = pd.DataFrame(data)

# Inspect
print(df.head())
print(df.shape)
df.info()

# Clean text
df["Name"] = df["Name"].str.strip().str.title()
df["Department"] = df["Department"].str.strip().str.upper()

# Convert numeric data
df["Marks"] = pd.to_numeric(df["Marks"], errors="coerce")

# Check and handle missing values
print(df.isnull().sum())
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
df["CGPA"] = df["CGPA"].fillna(df["CGPA"].mean())

# Remove duplicates
df = df.drop_duplicates()

# Validate ranges
invalid_marks = df[
    (df["Marks"] < 0) | (df["Marks"] > 100)
]
invalid_cgpa = df[
    (df["CGPA"] < 0) | (df["CGPA"] > 4)
]

print("Invalid Marks:")
print(invalid_marks)
print("Invalid CGPA:")
print(invalid_cgpa)

# Transform
df["percentage"] = df["CGPA"] / 4 * 100

df["status"] = np.where(
    df["Marks"] >= 50,
    "Pass",
    "Fail"
)

df["grade"] = np.where(
    df["Marks"] >= 90, "A+",
    np.where(
        df["Marks"] >= 80, "A",
        np.where(
            df["Marks"] >= 70, "B",
            np.where(
                df["Marks"] >= 60, "C",
                "F"
            )
        )
    )
)

```

```
)

# Sort and display
df = df.sort_values(
    by="Marks",
    ascending=False
)

print(df)
```

This project demonstrates the full workflow: inspect → standardize text → convert numeric data → identify missing values → fill missing values → remove duplicates → validate ranges → create derived columns → sort → inspect the cleaned result.

## 17. Project Analysis

```
average_marks = df["Marks"].mean()
average_cgpa = df["CGPA"].mean()
highest_marks = df["Marks"].max()

print("Average Marks:", average_marks)
print("Average CGPA:", average_cgpa)
print("Highest Marks:", highest_marks)

top_student = df.loc[df["Marks"].idxmax()]
print(top_student)

print(df["Department"].value_counts())
print(df["grade"].value_counts())

print("Missing values:")
print(df.isnull().sum())

print("Duplicate rows:")
print(df.duplicated().sum())
```

idxmax() finds the index of the maximum value. loc[] retrieves the complete row. value\_counts() counts category frequencies. The final checks verify data quality.

## 18. Practice Tasks

**Task 1:** Create a DataFrame with name, marks, and CGPA where at least one value is missing. Count missing values.

**Task 2:** Fill missing Marks with the mean.

**Task 3:** Fill missing CGPA with the median.

**Task 4:** Create duplicate student records and remove duplicates.

**Task 5:** Clean names such as ' ali ', ' usman ', and ' HAMZA ' into consistent title case.

**Task 6:** Standardize department values such as cs, CS, Cs, se, and SE.

**Task 7:** Convert ['85', '90', 'invalid', '75'] into numeric values using pd.to\_numeric() and errors='coerce'.

**Task 8:** Find marks below 0 or above 100.

**Task 9:** Filter students whose Marks are at least 80.

**Task 10:** Create a Pass/Fail column where Marks >= 50 is Pass and Marks < 50 is Fail.

## 19. Complete Solutions

### Solution 1

```
import pandas as pd
data = {
    "name": ["Ali", "Usman", "Hamza"],
    "marks": [80, None, 90],
    "cgpa": [3.2, 3.0, None]
}
df = pd.DataFrame(data)
print(df.isnull().sum())
```

### Solution 2

```
df["marks"] = df["marks"].fillna(df["marks"].mean())
```

### Solution 3

```
df["cgpa"] = df["cgpa"].fillna(df["cgpa"].median())
```

### Solution 4

```
df = df.drop_duplicates()
```

### Solution 5

```
df["name"] = df["name"].str.strip().str.title()
```

### Solution 6

```
df["department"] = df["department"].str.strip().str.upper()
```

### Solution 7

```
df["marks"] = pd.to_numeric(df["marks"], errors="coerce")
```

### Solution 8

```
invalid = df[(df["marks"] < 0) | (df["marks"] > 100)]  
print(invalid)
```

### Solution 9

```
high_performers = df[df["marks"] >= 80]  
print(high_performers)
```

### Solution 10

```
df["status"] = np.where(df["marks"] >= 50, "Pass", "Fail")
```

## 20. Common Mistakes

**Deleting all missing rows immediately:** You may lose useful information. Decide whether filling, dropping, or another strategy is appropriate.

**Using mean for every missing value:** Mean, median, mode, fixed values, and row removal have different use cases.

**Forgetting parentheses in conditions:** Use (condition1) & (condition2).

**Using and/or with Pandas Series:** Use & and | for element-wise conditions.

**Converting dirty numeric text directly:** Use pd.to\_numeric(..., errors='coerce') when invalid text may exist.

**Ignoring duplicates:** Duplicates can distort counts and statistics.

**Inconsistent text cleaning:** Standardize spaces and capitalization before grouping or filtering.

## 21. Quick Revision

| Command           | Purpose                         |
|-------------------|---------------------------------|
| df.isnull()       | Find missing values             |
| df.isnull().sum() | Count missing values            |
| df.notnull()      | Find available values           |
| fillna()          | Fill missing values             |
| dropna()          | Remove rows with missing values |
| duplicated()      | Detect duplicate rows           |
| drop_duplicates() | Remove duplicates               |
| rename()          | Rename columns                  |
| .str.strip()      | Remove surrounding spaces       |
| .str.lower()      | Lowercase text                  |
| .str.upper()      | Uppercase text                  |
| .str.title()      | Title case text                 |

| Command                      | Purpose                     |
|------------------------------|-----------------------------|
| <code>replace()</code>       | Replace inconsistent values |
| <code>astype()</code>        | Change data type            |
| <code>pd.to_numeric()</code> | Convert to numeric          |
| <code>sort_values()</code>   | Sort rows                   |
| <code>value_counts()</code>  | Count categories            |
| <code>np.where()</code>      | Conditional transformation  |

## 22. Final Checklist

- What Data Cleaning means
- Why data quality matters
- Inspecting a DataFrame
- Detecting and counting missing values
- `isnull()` and `notnull()`
- `fillna()` with mean, median, mode, or fixed values
- `dropna()`
- `duplicated()` and `drop_duplicates()`
- Renaming and standardizing columns
- `.str.strip()`, `lower()`, `upper()`, `title()`
- `replace()`
- `astype()` and `pd.to_numeric()`
- `errors='coerce'`
- Detecting invalid Marks and CGPA
- Filtering and sorting
- Creating new columns
- Basic transformations
- Complete cleaning and validation workflow

## 23. Final Takeaway

Day 13 taught you how to work with Pandas DataFrames. Day 14 teaches you how to make those DataFrames trustworthy. This cleaning mindset will be used repeatedly in Data Analysis, Data Visualization, Statistics, and Machine Learning.

```

RAW DATA
↓
INSPECT
↓
MISSING VALUES
↓
DUPLICATES
↓
TEXT CLEANING
↓
DATA TYPES
↓
INVALID VALUES
↓
TRANSFORMATION
↓
VALIDATION
↓

```

CLEAN DATA



ANALYSIS